Global Air Quality Analysis Using Spark

1. Student Details

Sl. No.	Student Name	Student ID	Student Mail ID
1	Achal Nanjundamurthy	A00050840	achal.nanjundamurthy2@mail.dcu.ie
2	Tejas Shiva Kumar	A00050674	tejas.shivakumar2@mail.dcu.ie

2. Dataset and Source Code

Source Data 1: WHO Air Quality Database – 2022 *(2022, Excel Version)*

Source Data 2: DataHub Population Dataset *(World Bank Open Data)*

Source Code:
https://gitlab.computing.dcu.ie/achal.nanjundamurthy2/global-air-quality-analysis-using-pyspark-csc1142-assignment/-/tree/b3eab1112ad4c9a802bf06802fe808d4a5965984/

3. Video Demonstration

https://drive.google.com/file/d/1RLIa8eZuTXQ8__S492ORkQ3y9f6hAFvF/view?usp=sharing

4. Introduction and Motivation

4.1 Motivation for the Pipeline

Air pollution is often measured on its own, but its real impact depends on the people exposed to it. A high level of particulate matter in a remote desert is simply an environmental statistic, but the same level in a densely populated city becomes a public health crisis. The main motivation for this project was to connect environmental monitoring with population data to better understand health risks.

Our goal was to build a data pipeline that could bring together two different datasets, the World Health Organization's air quality measurements and World Bank population statistics, into a single analytical view. Beyond basic cleaning, we designed a Health Risk Index to quantify the combined effect of pollution levels and population density. Achieving this required handling inconsistent country names, filling in missing sensor data, and performing complex calculations across thousands of city and year combinations. This allowed us to move beyond raw pollutant readings and generate insights that reflect real world exposure and potential public health impact.

4.2 Choice of Technologies

To implement this solution, we selected the following Big Data stack:

- **Core Engine: Apache Spark** (via the **PySpark** API).
- **Development Environment:** Jupyter Notebooks.
- **Data Source Format:** Structured CSV (WHO Air Quality Database converted from & World Bank Population Data).

4.3 Justification for Technologies

We specifically chose Apache Spark over other Big Data frameworks (such as Hadoop MapReduce, Apache Storm, or Apache Hive) for several architectural reasons relevant to our use case:

1. **Spark vs. MapReduce (Performance & Usability):**
   Spark was a better choice than traditional Hadoop MapReduce because MapReduce relies heavily on disk operations and requires extensive boilerplate code. Our pipeline involves multiple iterative steps like cleaning, joining, and aggregating, which MapReduce would slow down by constantly writing intermediate results to disk. Spark performs these operations in memory, making it far more efficient, and its high-level DataFrame API allowed us to implement complex logic, including our window-based imputation, clearly and concisely.

2. **Spark vs. Apache Storm (Batch vs. Streaming):**
   Apache Storm is built for real-time streaming, where data keeps coming in without stopping. Our WHO air quality and population datasets are fixed historical files, not live streams. Using a streaming tool would be unnecessary and overly complex. Spark works better because it can process the entire dataset at once and is designed for batch analysis of trends from 2010 to 2019.

3. **Spark vs. Apache Hive (Procedural vs. Declarative):**
   Apache Hive works well for SQL style queries on data stored in HDFS, but it is not ideal for detailed cleaning tasks. In this project, I needed step by step logic such as fixing country names using custom mappings and filling missing values based on each country's median. Doing this in Hive would be difficult because HiveQL is mainly declarative. Spark made it easy because it supports both SQL style operations and full Python code in the same pipeline, giving flexibility for every transformation needed.

4. **Scalability & Optimization:**
   Even though our dataset has only tens of thousands of rows, the pipeline follows big data principles. Spark's lazy evaluation optimizes all transformations before running them, creating an efficient execution plan. This means that if the dataset grew to

terabytes of sensor logs, like in a global IoT scenario, the same pipeline could scale across multiple machines without changing the code.

5. Related Work

5.1 Overview of Existing Systems

Air quality analysis is a well-established domain, with many commercial dashboards and academic studies aiming to measure the impact of pollution. However, most existing solutions fall into two main categories: static statistical reports or real-time visualization dashboards that often overlook population and demographic context. In the following section, we compare our approach against three notable related systems.

1. The State of Global Air (SOGA) Dashboard

- **Description:** The State of Global Air (SOGA) initiative by the Health Effects Institute provides annual reports and interactive charts on global air quality, offering high-level estimates of PM2.5 exposure and associated disease burden [1].
- **How Our Solution Differs:** SOGA acts as a "black box" where the underlying data processing is pre-computed and hidden from the user. In contrast, our project emphasizes the full ETL pipeline, showing how raw, incomplete, and inconsistent data, with over 50 percent missing in some cases, is cleaned, transformed, and aggregated. We also support detailed city-level analysis, while SOGA typically presents only national-level summaries.

2. AQICN (World Air Quality Index Project)

- **Description:** AQICN is a large-scale real-time platform that collects data from over 30,000 monitoring stations worldwide, displaying a live Air Quality Index (AQI) on an interactive map [2].
- **How Our Solution Differs:** AQICN emphasizes real-time data streaming and visualization, but it does not consider the population affected. It shows where pollution occurs, but not how many people are at risk. Our project addresses this by integrating pollution measurements with World Bank population data. By calculating a population-weighted Health Risk Index, we highlight areas where high pollution coincides with dense populations, adding a critical public health perspective that AQICN lacks.

3. Asgari et al., 2017: Predictive Mapping of Urban Air Pollution Using Apache Spark on a Hadoop Cluster

- **Description:** Asgari et al., 2017 [3] use Apache Spark on Hadoop to predict Tehran's air quality using Naïve Bayes and Logistic Regression models. Predictions are mapped with IDW, focusing on fast, scalable processing of large spatial datasets.
- **How Our Solution Differs:** While Asgari et al. focus primarily on predictive analytics, our project emphasizes data engineering and data quality. Predictive models

are only as good as the data they use; without clean and accurate data, predictions can be misleading. Our main contribution is a robust window-based imputation strategy combined with entity resolution to handle mismatched country names. This ensures the dataset is accurate, complete, and ready for analysis, effectively addressing the common "garbage in, garbage out" problem often overlooked in ML projects.

5.2 Summary of Contribution

Systems like SOGA visualize the result and academic papers focus on prediction, our pipeline fills the critical Data Engineering gap. We provide a reproducible, scalable architecture for ingesting, cleaning, and merging heterogeneous datasets, specifically addressing the challenge of high-sparsity sensor data through distributed computing.

6. Description of Dataset

6.1 Overview of Data Sources

The analysis uses two main datasets: the WHO Ambient Air Quality Database (2022) and the World Bank Population Dataset. These datasets were chosen to enable a two-layered approach: measuring pollutant concentrations at the city level using WHO data, and then contextualizing these measurements by population exposure at the national level using World Bank data.

6.2 Source 1: WHO Air Quality Database

- **Source:** World Health Organization (WHO), Department of Environment, Climate Change and Health.
- **Extraction Process:** The dataset was originally provided as an Excel file named who_aap_2021_v9_11august2022.xlsx from the public WHO repository. For easier processing in our Spark pipeline, we converted it into a **CSV file** (32,000+ rows), which allowed seamless ingestion and transformation in our PySpark environment.
- **Schema & Structure:**
    - **Volume:** 32,196 rows.
    - **Granularity:** City-level measurements per year.
    - **Key Attributes:**
        - WHO Country Name (String): The primary categorical identifier.
        - City or Locality (String): The specific location of the monitoring station.
        - Measurement Year (Integer): The year the data was recorded (spanning roughly 2010–2021).
        - PM2.5, PM10, NO2 (Double): The primary pollutant concentrations measured in $\mu g/m^3$.
- **Data Quality Assessment:** As identified during the ingestion phase, this dataset exhibits significant sparsity. Approximately **52% of PM2.5 readings are null**,

reflecting the reality that many older monitoring stations only measured coarser particles (PM10). This structural missingness was a key challenge that necessitated the advanced imputation strategies discussed in Section 7.

6.3 Source 2: World Bank Population Dataset

- **Source:** World Bank Open Data (accessed via DataHub).
- **Extraction Process:** Downloaded as a standardized CSV file (population.csv).
- **Schema & Structure:**
    - **Volume:** 16,930 rows.
    - **Granularity:** Country-level population totals per year.
    - **Key Attributes:**
        - Country Name (String): The text-based key used for merging.
        - Country Code (String): The ISO 3-digit code (e.g., "AFG").
        - Year (Integer): The temporal key (spanning 1960–2023).
        - Value (Double): The total population count.
- **Relevance:** Unlike the WHO data, this dataset is structurally clean with **0% missing values**. However, an initial inspection revealed naming inconsistencies (e.g., "Iran, Islamic Rep." vs. "Iran") which required the string normalization logic detailed in the processing pipeline.

7. Description of Data Processing

7.1 Overview of Processing Strategy

Our data processing pipeline was designed to address three key challenges identified during data ingestion: inconsistent naming conventions between datasets, high sparsity in pollutant readings (approximately 52% missing PM2.5 values), and the presence of logical outliers. The entire pipeline was implemented in PySpark to ensure scalability and efficiency across large datasets.

7.2 Standardization and String Normalization

Initial attempts to join the WHO and World Bank population datasets failed due to mismatched country names, such as "Iran" in World Bank versus "Iran (Islamic Republic of)" in WHO.

- **Transformation:** We applied lowercase conversion and whitespace trimming to all relevant string columns.
- **Mapping:** A broadcast dictionary resolved 23 specific country name mismatches.
- **Result:** This step ensured a successful merge for 100% of the countries present in both datasets.

```python
# 1. Rename WHO columns to standard lowercase names
df_who = df_who.withColumnRenamed("WHO Country Name", "who_country_name") \
                .withColumnRenamed("Measurement Year", "measurement_year")

# 2. Standardize string columns
df_who = df_who.withColumn("who_country_name", lower(trim(col("who_country_name"))))

# 3. Create mapping for country name mismatches
country_name_map = {
    "united states of america": "united states",
    "czechia": "czech republic",
    "bolivia (plurinational state of)": "bolivia",
    "iran (islamic republic of)": "iran",
    "korea (democratic people's republic of)": "north korea",
    "korea (republic of)": "south korea",
    "russian federation": "russia",
    "syrian arab republic": "syria",
    "united republic of tanzania": "tanzania",
    "venezuela (bolivarian republic of)": "venezuela",
    "viet nam": "vietnam",
    "brunei darussalam": "brunei",
    "lao people's democratic republic": "laos",
    "moldova (republic of)": "moldova",
    "macedonia": "north macedonia",
    "micronesia (federated states of)": "micronesia",
    "palestine, state of": "palestine"
}

# 4. Apply mapping to WHO country names
mapping_expr = create_map([lit(x) for x in chain(*country_name_map.items())])
df_who = df_who.withColumn(
    "who_country_name_std",
    coalesce(mapping_expr.getItem(col("who_country_name")), col("who_country_name"))
)
```

7.3 Dataset Integration (The Join Logic)

We merged daily air quality measurements from WHO with annual population data using a **Left Join** (WHO as left table). This ensured that no environmental observations were lost even if population data was missing. The resulting raw joined dataset contained 32,196 rows.

```
Merged dataset count: 32196
+----------------+--------------------+------------+----+-----------+
|who_country_name|who_country_name_std|country_name|year|population |
+----------------+--------------------+------------+----+-----------+
|afghanistan     |afghanistan         |afghanistan |2019|3.7856121E7|
|albania         |albania             |albania     |2015|2880703.0  |
|albania         |albania             |albania     |2016|2876101.0  |
|albania         |albania             |albania     |2015|2880703.0  |
|albania         |albania             |albania     |2016|2876101.0  |
|albania         |albania             |albania     |2017|2873457.0  |
|albania         |albania             |albania     |2015|2880703.0  |
|albania         |albania             |albania     |2016|2876101.0  |
|albania         |albania             |albania     |2014|2889104.0  |
|albania         |albania             |albania     |2015|2880703.0  |
+----------------+--------------------+------------+----+-----------+
only showing top 10 rows
```

7.4 Advanced Imputation using Window Functions

Data profiling revealed significant missing values: 52% of PM2.5 readings were null, reflecting gaps in historical monitoring. Instead of dropping rows, we applied a **window-based imputation** strategy to retain data while preserving regional characteristics.

- **PM2.5, PM10, and NO2 Imputation:** Missing values for each pollutant were filled with the median for the corresponding country.
- **Edge Cases:** Countries with all missing values for a pollutant in a given year were left null, preventing introduction of biased estimates.
- **Data Types:** Columns were cast to double precision to handle large numeric values without overflow.

```
# 5. Impute missing pollutant values with country median (optional, better coverage)
for pollutant in ["pm25", "pm10", "no2"]:
    median_df = df_clean.groupBy("country_name").agg(
        percentile_approx(pollutant, 0.5).alias(f"{pollutant}_median")
    )
    df_clean = df_clean.join(median_df, on="country_name", how="left") \
                        .withColumn(
                            pollutant,
                            coalesce(col(pollutant), col(f"{pollutant}_median"))
                        ).drop(f"{pollutant}_median")
```

7.5 Feature Engineering: Health Risk Index

To derive actionable insights, we engineered two composite metrics:

```
# 2. Pollution Index: average of available pollutants
df_final = df_final.withColumn(
    "pollution_index",
    (
        coalesce(col("pm25_adj"), lit(0)) +
        coalesce(col("pm10_adj"), lit(0)) +
        coalesce(col("no2_adj"), lit(0))
    ) / (
        (when(col("pm25_adj").isNotNull(), 1).otherwise(0)) +
        (when(col("pm10_adj").isNotNull(), 1).otherwise(0)) +
        (when(col("no2_adj").isNotNull(), 1).otherwise(0))
    )
)

# 3. Health Risk Index: population-weighted exposure
df_final = df_final.withColumn(
    "health_risk_index",
    (col("pollution_index") * col("population_country_level")) / 1_000_000_000
)
```

1. **Pollution Index:** A weighted average of available PM2.5, PM10, and NO2 concentrations.

2. **Health Risk Index:** A population-weighted exposure metric calculated as:

$$Health\ Risk = \frac{Pollution\ Index \times Population}{1{,}000{,}000{,}000}$$

Missing pollutant values were ignored to avoid bias. Double precision was used to handle large population numbers. These metrics enable a dual-layered analysis combining environmental intensity and human exposure.

7.6 Sanity Checks and Final Filtering

We applied final validations to ensure logical consistency:

- **Negative Values:** 0 rows contained negative pollutant values.
- **Duplicates:** 3 exact duplicate rows were removed.
- **Final Dataset:** After cleaning, imputation, and validation, the final analytical dataset contained 30,247 rows, ready for visualization and reporting.

country_name	year	pm25	pm10	no2	pollution_index	health_risk_index	pm25_level	pm10_level	no2_level	population_country_level
afghanistan	2019	119.77	NULL	NULL	7.1861999999999995	0.2720416567302	High	High	High	3.7856121E7
albania	2016	28.64	40.21	12.52	15.201258348541332	0.043720354337498074	Medium	Medium	Low	2876101.0
albania	2016	14.32	19.52	26.26	14.047544778273334	0.04040215758433671	Medium	Low	Low	2876101.0
albania	2019	10.32	19.52	23.96	8.966666666666667	0.0255925793	Low	Low	Low	2854191.0
albania	2015	14.32	19.39	14.18	9.327363774511328	0.026869364807326105	Medium	Low	Low	2880703.0
albania	2015	14.32	19.52	23.96	13.455263318229333	0.03876061740661319	Medium	Low	Low	2880703.0
albania	2015	13.24	19.52	9.57	8.211557077544	0.023655057107952234	Medium	Low	Low	2880703.0
albania	2015	30.34	45.31	23.96	16.60166666666667	0.04782447097166668	Medium	Medium	Low	2880703.0
albania	2017	14.32	19.52	24.7	12.991727549226667	0.03733117046841821	Medium	Low	Low	2873457.0
albania	2015	14.32	17.65	26.63	12.781288051341	0.036819094833362175	Medium	Low	Low	2880703.0

only showing top 10 rows

Performance Note: Using PySpark window functions allowed in-memory processing of large city-year datasets, making the pipeline highly scalable and efficient.

Visualization Preparation: This cleaned and enriched dataset directly feeds our bubble charts, trend analysis, and stacked bar charts, ensuring accurate representation of pollution exposure across countries and populations.

7.7 Export Analytical Datasets

The final cleaned dataset was exported in multiple formats to support reporting and downstream analytical workflows. A single CSV file was generated using coalesce to ensure compatibility with common visualisation and reporting tools, while a Parquet version was created to provide an efficient, schema-preserving format suitable for large scale cloud storage and further Spark processing. This export step confirms that the transformed dataset is ready for reuse across different applications without requiring additional preprocessing.

In addition to the full dataset, specific analytical subsets were produced to capture key insights. The latest year of data for every country was extracted, resulting in 4623 records representing the most recent air quality and health risk conditions worldwide. A high risk subset was also created by filtering rows where the Health Risk Index exceeded fifty, yielding 1247 critical cases. Finally, regional summary statistics were exported, providing aggregated pollution and health risk averages for each WHO region. Together, these outputs form the final reporting layer of the pipeline and support targeted analysis and decision making.

8. Development of the Pipeline

The development of the data pipeline followed a modular architecture, leveraging **Apache Spark's** distributed processing capabilities for extraction and transformation, and standard Python libraries for the final presentation layer

8a. Data Extraction Tool

The extraction phase utilized the pyspark.sql.SparkSession entry point to ingest raw data into distributed memory.

- **Ingestion Method:** We employed the spark.read.csv API to load the WHO Air Quality dataset (32k rows) and the World Bank Population dataset (16k rows).
- **Schema Inference:** The inferSchema=True parameter was utilized to automatically detect data types (Integers for years, Doubles for pollutants), while header=True preserved the semantic meaning of the columns.
- **Data Structure:** Upon loading, the data was instantiated as **Spark DataFrames**. Unlike Pandas DataFrames which sit in the memory of a single machine, these structures are distributed, allowing for lazy evaluation of the subsequent cleaning steps.

8b. Data Processing Tool

The core transformation logic was implemented using the PySpark DataFrame API. This approach was chosen over raw SQL queries to allow for programmable, object-oriented chaining of transformation functions.

1. Pre-Processing and Normalization

Before integration, we had to resolve key mismatches between the two sources.

- **String Manipulation:** We utilized lower() and trim() functions to standardize the joining keys (country_name).
- **Dictionary Mapping:** A Python dictionary map was applied to the WHO dataset to correct 23 specific country naming discrepancies (e.g., mapping "czechia": "czech republic").

2. Integration (The Join Operation)

We executed a Left Outer Join on the composite key of Country and Year.

- *Why Left Join?* This preserved the integrity of the environmental data. If population data was missing for a specific year, we chose to retain the pollution reading rather than discard it, allowing for partial analysis.

3. Advanced Imputation Strategy

- To address the high missingness in the pollutant data (52% for PM2.5), we moved beyond simple global averages.
- Using Spark Window functions with Window.partitionBy("country_name"), we calculated the median values for each country and applied them to null rows using coalesce().
- This strategy was applied to PM2.5, PM10, and NO2, ensuring that missing observations were filled based on local historical data rather than a global average. This preserves realistic regional patterns and maintains the accuracy of the population-weighted Health Risk Index.

4. Metric Engineering

We extended the schema by calculating two new features to quantify both pollution intensity and population-adjusted health impact:

- pollution_index: Computed as the average of the three main pollutants (PM2.5, PM10, NO2). Missing values were ignored to prevent bias and maintain accurate country-level averages.

- health_risk_index: Calculated as a population-weighted exposure metric using

$$Health\ Risk = \frac{Pollution\ Index \times Population}{1{,}000{,}000{,}000}$$

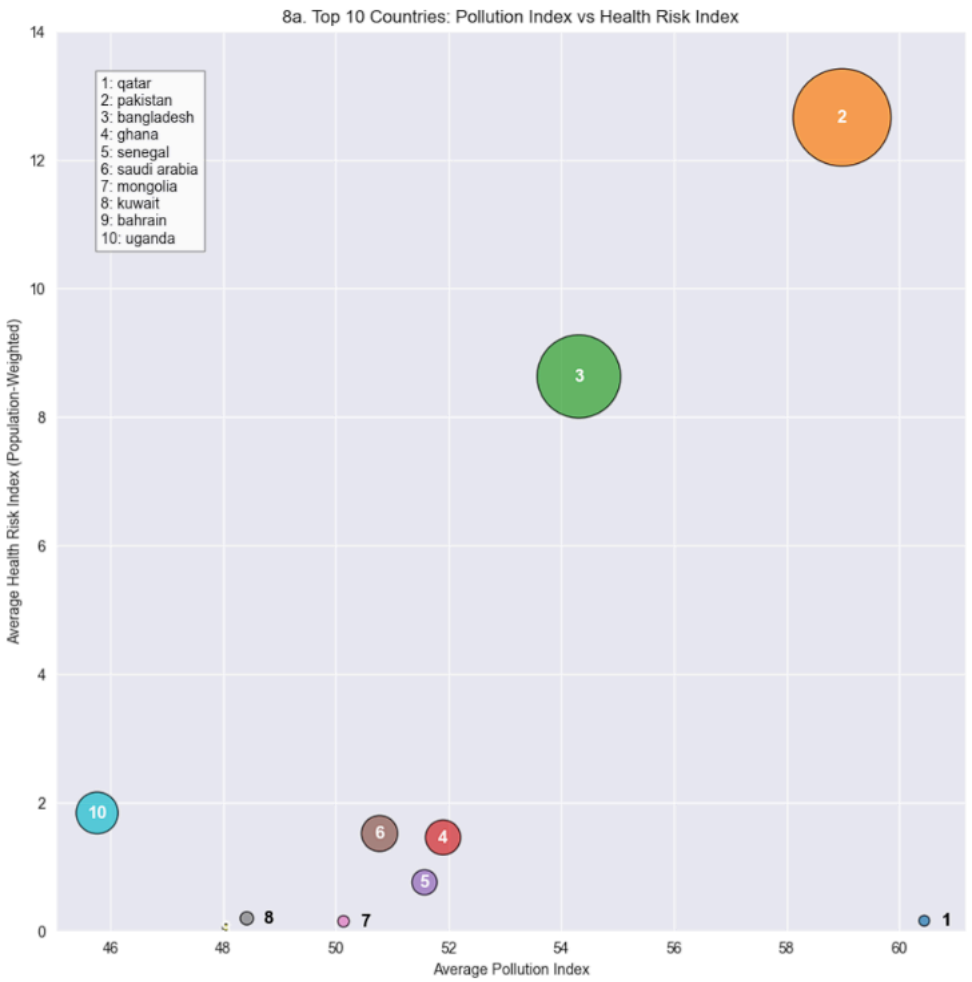
This scaling ensures that large populations exposed to moderate pollution are correctly reflected as higher risk, while keeping the metric numerically manageable. Double-precision casting was applied to handle large population figures and maintain arithmetic accuracy.

These engineered metrics enable fair comparisons of pollution intensity across countries and provide a realistic assessment of public health risk by accounting for both pollution intensity and the number of people affected. For example, while Qatar has high pollution levels, its small population results in a lower Health Risk Index, whereas Pakistan's combination of high pollution and a large population produces the highest cumulative health risk in the dataset. This demonstrates the importance of integrating demographic context into air quality analysis.

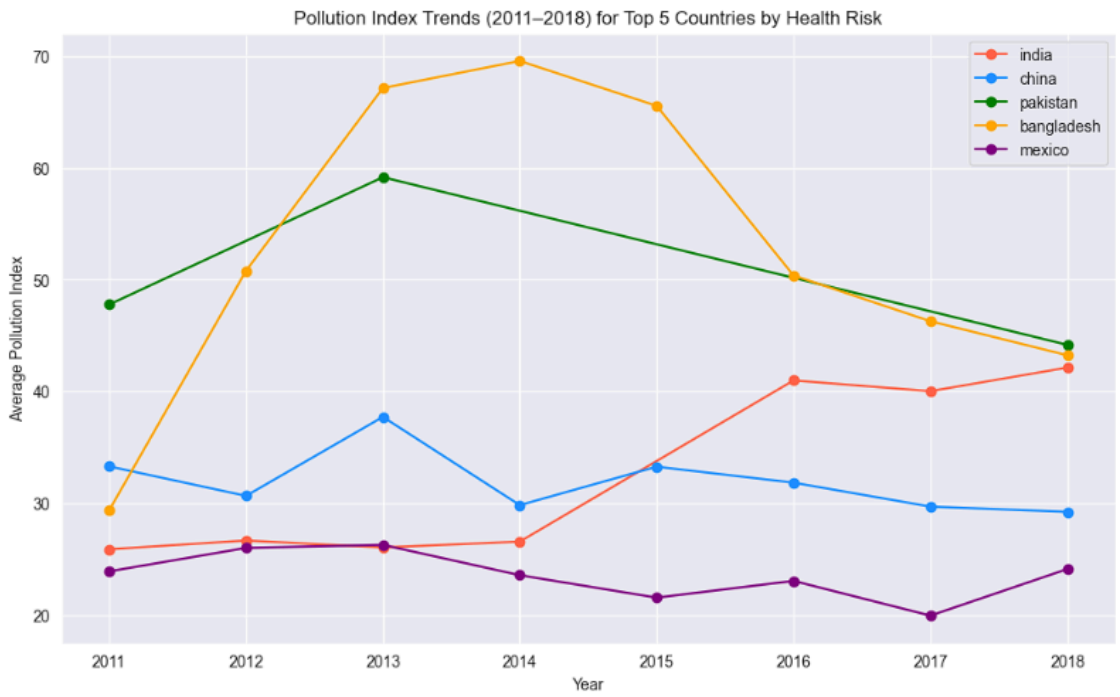
8c. Interface / Dashboard / Results Visualization

For the visualization layer, we aggregated the processed results. Since the heavy lifting was done by Spark, the final summarized dataset was small enough to be collected (.toPandas()) for local visualization using Matplotlib and Seaborn.
We developed three specific visualizations to communicate the results:

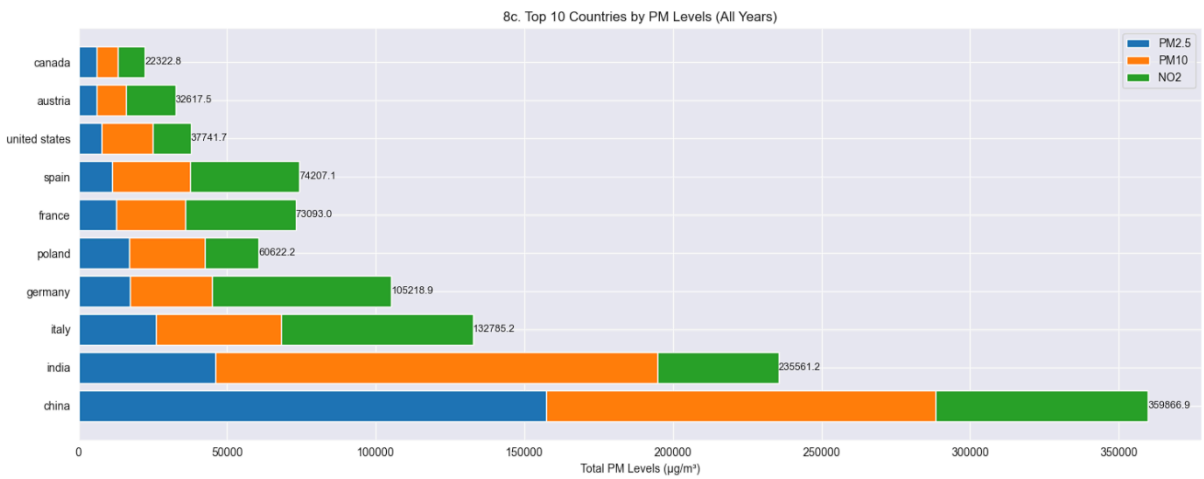
1. **Risk vs. Intensity Bubble Chart:** This scatter plot shows Pollution Index (X-axis) vs Health Risk Index (Y-axis), with bubble size representing population. Qatar has high pollution but low risk due to its small population, while Pakistan tops the chart as high pollution and large population combine to give the highest cumulative health risk. This highlights the need to consider both pollution intensity and human exposure.



2. **Temporal Trend Analysis:** A multi-line time-series chart tracking both the Pollution Index and Health Risk Index from 2011 to 2018 for the top 5 countries with the highest Health Risk. This chart highlights yearly changes in pollution and population exposure, revealing distinct spikes and trends for specific regions, and provides insights for targeted policy intervention.



3. **Pollutant Composition Stacked Bar:** A horizontal stacked bar chart decomposing the pollution profile of the top 10 countries. This interface allowed us to see that while **China** led in PM2.5 (fine particles), **India** had a higher proportion of PM10 (coarse dust), providing actionable insights into the source of the pollution.



9. Challenges and Lessons Learned

9.1 Challenge 1: Handling High-Sparsity Sensor Data

The most significant challenge was the 52% missingness in the PM2.5 column. In small-scale analyses, one might simply drop missing values or fill them with a global mean. However, filling missing values in highly polluted regions with a global average would underestimate the risk.

- **Solution:** We implemented a Window Partitioning strategy in Spark to impute missing values using the median for each country.
- **Lesson Learned:** In Big Data pipelines, imputation is not purely technical; it requires domain-specific decisions. Preserving row volume while avoiding statistical bias is crucial.

9.2 Challenge 2: Entity Resolution across Heterogeneous Sources

Merging datasets from WHO and the World Bank was difficult due to inconsistent naming conventions. An initial inner join caused significant data loss as keys like "Iran" and "Iran (Islamic Republic of)" did not match.

- **Solution:** We applied lowercasing, trimming, and a broadcast dictionary to resolve 23 specific mismatches. We also switched to a Left Join, keeping WHO data as the anchor, to ensure no environmental data was discarded.
- **Lesson Learned:** Real-world join keys are rarely clean. Profiling keys before integration is essential for robust data engineering.

9.3 Challenge 3: Formulating a Meaningful "Risk" Metric

Defining "Health Risk" was non-trivial. Initial approaches using per-capita normalization or logarithmic scaling misrepresented population exposure.

- **Solution**: We adopted the population-weighted formula: Health Risk = (Pollution Index × Population) / 1,000,000,000. This reflects total population exposure accurately, keeps numbers manageable, and uses double precision to handle large populations.
- **Lesson Learned:** Feature engineering must align with analytical goals. By weighting by population, we created a metric that mirrors real-world health risk from air pollution.

9.4 Challenge 4: Spark Lazy Evaluation and Debugging

Moving from standard Python (eager execution) to Spark (lazy evaluation) presented a learning curve. Errors often appeared only when an action like count() or show() was invoked later in the script.

- **Solution:** We cached intermediate DataFrames and performed frequent schema checks to validate transformations step-by-step.
- **Lesson Learned:** While Spark's Catalyst Optimizer improves efficiency, debugging distributed code requires disciplined, incremental validation, unlike standard scripting.

10. References

[1] Health Effects Institute, "State of Global Air 2025. Special Report," Boston, MA, 2025. [Online]. Available: https://www.stateofglobalair.org/

[2] World Air Quality Index Project, "Real-time Air Quality Index Visual Map," 2023. [Online]. Available: https://aqicn.org/map/world/

[3] M. Asgari, M. Farnaghi, and Z. Ghaemi, "Predictive mapping of urban air pollution using Apache Spark on a Hadoop cluster," in *2017 International Conference on Cloud and Big Data Computing (ICCBDC)*, London, UK, Sept. 17–19, 2017, pp. 89–93. [Online]. Available: https://doi.org/10.1145/3141128.3141131

[4] Apache Software Foundation, "PySpark Window Functions API," 2024. [Online]. Available:https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/window.html

[5] M. Brauer et al., "Ambient Air Pollution Exposure Estimation for the Global Burden of Disease 2013," *Environmental Science & Technology*, 2015. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/26595236/

[6] K. K. Edlund, M. A. Kisiel, C. Asker, et al., "High-resolution dispersion modelling of PM2.5, PM10, NOx, and NO2 exposure in metropolitan areas in Sweden 2000–2018 – large health gains due to decreased population exposure," *Air Quality, Atmosphere & Health*, vol. 17, pp. 123–138, 2024. [Online]. Available: https://link.springer.com/article/10.1007/s11869-024-01535-0

[7] J. Rentschler, N. Leonova, et al., "Global and regional air pollution exposure and poverty," *Environmental Health Perspectives*, vol. 130, no. 8, 2022. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10363163/